TROUBLESHOOTING HANDBOOK

VC

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Troubleshooting Handbook

End-to-end troubleshooting procedures for VCF components including SDDC Manager, vCenter, ESXi, and NSX.

SDDC MANAGER | VCENTER | ESXI | NSX | END-TO-END

VCF 9.0 VMware Cloud Foundation

VMware Cloud Foundation 9.0 Nested Deployment Troubleshooting Handbook

Version: 1.6 **Date:** May 2026 **Environment:** VCF 9.0.1 Nested in VMware Workstation **Author:** Virtual Control LLC

> ***PowerShell Operators:*** *Every workstation-runnable bash code block in this handbook now ships with a paste-ready PowerShell sibling intended for Windows admin workstations (PS 5.1 or 7+). Translation conventions (* `curl.exe` *,* `Invoke-RestMethod` *, here-strings,* `ConvertFrom-Json` *, cert-bypass, etc.) are documented in* `../POWERSHELL-TRANSLATION-REFERENCE.md` *. Bash blocks that run inside an appliance (VCF Installer / SDDC Manager / vCenter / NSX / ESXi) —* `service-control` *,* `systemctl` *,* `journalctl` *,* `psql` *,* `vmon-cli` *,* `keytool` *,* `vim-cmd` *,* `esxcli` *,* `vmkfstools` *,* `partedUtil` *,* `vdq` *,* `services.sh` *, on-host file ops under* `/etc` *,* `/var/log` *,* `/storage` *,* `/tmp` *, and* `/data` *— intentionally have* ***no*** *PowerShell sibling; SSH into the appliance and run them there.*

Revision History

Version	Date	Author	Changes
1.5	February 2026	Virtual Control LLC	NSX cert SAN/Trust playbooks (7.2.1, 7.2.2, 9.6); VCF Ops health-adapter flows (9.7, 9.8); SDDC Manager credential cascade DB repair (Section 10); SDDC Manager local→vSAN migration (Section 11).
1.6	May 2026	Virtual Control LLC	Added paste-ready PowerShell sibling blocks for every workstation-runnable bash example; added cross-link to `../POWERSHELL-TRANSLATION-REFERENCE.md` . Appliance-only blocks (`service-control` , `systemctl` , `journalctl` , `psql` , `vmon-cli` , `keytool` , `vim-cmd` , `esxcli` , `vmkfstools` , `partedUtil` , `vdq` , `services.sh` , on-host file ops under `/etc` , `/var/log` , `/storage` , `/tmp` , `/data`) annotated as appliance-only.

Table of Contents

1. Executive Summary

Where commands run in this handbook. Section headers throughout this doc carry parenthetical environment labels — for example *(VCF Installer)*, *(ESXi Host)*, *(VCF Installer)*, *(SDDC Manager)*. These map to the following standard environments:

- **(VCF Installer)** — SSH session to the Cloud Builder / VCF Installer appliance (Photon-based VM, port 22). Default user is the appliance root or admin.
- **(ESXi Host)** — SSH session to one of the inner ESXi hosts (esxi01-04). Use the host's root password from your credential vault. Commands use `esxcli`, `vmkfstools`, `vmware-cmd`.
- **(SDDC Manager)** — SSH session to the SDDC Manager appliance, OR its REST API via curl/Postman.
- **(vCenter)** — either browser UI at `https://<vcenter-fqdn>/ui/` OR SSH to the VCSA (root login enabled with `chsh -s /bin/bash root`).
- **(Workstation PowerShell)** — PowerShell on your Windows workstation with PowerCLI module installed (`Install-Module VMware.PowerCLI`).
- **(Browser)** — UI navigation; no shell.

When a command block has no parenthetical label, infer from content: `esxcli` / `vmkfstools` = ESXi Host shell; `Get-VM` / `Connect-VIServer` = PowerCLI on workstation; `curl` against `:443` API = any shell with network access. **Substitute your own credentials** — placeholders like `<your-password>` mean "do not paste the literal text; use your vault."

This handbook documents the complete troubleshooting process for deploying VMware Cloud Foundation 9.0.1 in a nested VMware Workstation environment. The deployment encountered several challenges including:

- **Offline Depot TLS Issues**: BouncyCastle FIPS TLS implementation rejecting self-signed certificates
- **ESXi Certificate Mismatches**: Hosts with incorrect hostname in SSL certificates
- **vSAN SSD Detection**: Virtual disks not being recognized as SSDs for vSAN cache tier
- **Network Configuration**: Proper VLAN and subnet configuration for nested environments
- **SSH Requirements**: vLCM host seeding requires SSH enabled on all ESXi hosts
- **vCenter Deployment Failures**: Stuck deployments, PostgreSQL initialization issues, service startup failures
- **Failed Deployment Recovery**: Complete cleanup procedures when deployments fail without rollback option

This document provides step-by-step remediation procedures and recovery processes for nested VCF deployments.

2. Environment Overview

2.1 Infrastructure Components

Component	FQDN	IP Address
VCF Installer/SDDC Manager	vcf-installer.lab.local	192.168.1.240
vCenter Server	vcenter.lab.local	192.168.1.69
NSX Manager VIP	nsx-vip.lab.local	192.168.1.70
NSX Manager Node 1	nsx-node1.lab.local	192.168.1.71
ESXi Host 1	esxi01.lab.local	192.168.1.74
ESXi Host 2	esxi02.lab.local	192.168.1.75
ESXi Host 3	esxi03.lab.local	192.168.1.76
ESXi Host 4	esxi04.lab.local	192.168.1.82
VCF Operations	vcf-ops.lab.local	192.168.1.77
Fleet Management	fleet.lab.local	192.168.1.78
Collector	collector.lab.local	192.168.1.79
VCF Automation	automation.lab.local	192.168.1.90
Automation Node 1	automation-node1.lab.local	192.168.1.91
Offline Depot Server	(IP only - no DNS)	192.168.1.160
DNS/NTP Server	(Windows Server)	192.168.1.230

2.2 Network Configuration

Network	VLAN ID	Subnet	Gateway	IP Range
ESX Management	0	192.168.1.0/24	192.168.1.1	DHCP/Static

VM Management	0	192.168.1.0/24	192.168.1.1	Same as ESX Mgmt
vMotion	100	192.168.100.0/24	192.168.100.1	192.168.100.10-20
vSAN	200	192.168.200.0/24	192.168.200.1	192.168.200.206-216
NSX TEP	300	192.168.250.0/24	192.168.250.1	192.168.250.10-25

2.3 ESXi VM Specifications (Nested)

Resource	Value
vCPUs	8 (4 cores x 2 sockets)
Memory	48 GB
OS Disk	32 GB
vSAN Cache Disk	100 GB (SSD)
vSAN Capacity Disk	800 GB (SSD)
Network Adapters	4x vmxnet3

3. Offline Depot Configuration

3.1 TLS/SSL Certificate Issues

Problem Description

VCF 9.0.1 uses BouncyCastle FIPS TLS implementation which has strict certificate validation requirements. When connecting to an offline depot with a self-signed certificate, the connection fails with:

```
Secure protocol communication error, check logs for more details
```

Symptoms

- VCF Installer UI shows "Secure protocol communication error"
- LCM debug logs show `TlsFatalAlert` errors:

```
org.bouncycastle.tls.TlsFatalAlert caught when processing request to {s}->https://192.168.1.160:8443
```

Root Cause

1. Self-signed certificate not trusted by Java keystore
2. Certificate may lack proper extensions for FIPS compliance
3. Certificate fingerprint mismatch between server and truststore

Diagnostic Commands (VCF Installer)

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Test SSL connectivity
openssl s_client -connect 192.168.1.160:8443

# Test with TLS 1.2 specifically
openssl s_client -connect 192.168.1.160:8443 -tls1_2

# Check cipher negotiation
openssl s_client -connect 192.168.1.160:8443 -tls1_2 </dev/null 2>&1 | grep -E "Cipher|Protocol|Verify"

# View certificate details
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509 -text -noout

# Get certificate fingerprint
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509 -noout -fingerprint -sha256

# Check LCM logs for TLS errors
grep -i "tlsfatal\|ssl\|certificate" /var/log/vmware/vcf/lcm/lcm-debug.log | tail -20

# Check LCM service status
systemctl status lcm
```

Appliance-only: *`openssl s_client` against the depot can also run from a Windows workstation, but the log inspection (`/var/log/vmware/vcf/lcm/`) and `systemctl` calls require SSH into the VCF Installer appliance. A workstation-side TLS probe is shown below for the connectivity test only.*

```
# PowerShell on Windows workstation -- TLS probe to depot only
# Connectivity / TLS / certificate inspection (no appliance-side log access)
$DepotHost = '192.168.1.160'
$DepotPort = 8443

# Force TLS 1.2 + 1.3 (skip cert validation for self-signed depot)
[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]'Tls12,Tls13'
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {
$true }

# Open a TLS stream and pull the server certificate
$tcp = New-Object System.Net.Sockets.TcpClient($DepotHost, $DepotPort)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({
$true }))
$ssl.AuthenticateAsClient($DepotHost)
$cert = New-Object
System.Security.Cryptography.X509Certificates.X509Certificate2($ssl.RemoteCerti

"Protocol : $($ssl.SslProtocol)"
"Cipher   : $($ssl.CipherAlgorithm) / $($ssl.HashAlgorithm)"
"Subject  : $($cert.Subject)"
"Issuer   : $($cert.Issuer)"
"NotAfter : $($cert.NotAfter)"
"SHA256   : $((Get-FileHash -InputStream
([IO.MemoryStream]::new($cert.RawData)) -Algorithm SHA256).Hash)"

$ssl.Dispose(); $tcp.Close()
```

3.2 Python HTTPS Server Setup

Server Script (C:\VCF-DEPOT\https_server.py)

```
#!/usr/bin/env python3
"""
HTTPS server for VCF Offline Depot
Serves files with TLS 1.2+ for SDDC Manager compatibility
"""
import http.server
import ssl
import os
```

```
import base64
import socketserver
from functools import partial

# Configuration
PORT = 8443
CERT_FILE = 'server.crt'
KEY_FILE = 'server.key'
USERNAME = 'admin'
PASSWORD = 'admin'

class AuthHandler(http.server.SimpleHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def __init__(self, *args, directory=None, **kwargs):
        super().__init__(*args, directory=directory, **kwargs)

    def do_HEAD(self):
        if not self.authenticate():
            return
        super().do_HEAD()

    def do_GET(self):
        if not self.authenticate():
            return
        super().do_GET()

    def do_POST(self):
        if not self.authenticate():
            return
        content_length = int(self.headers.get('Content-Length', 0))
        self.rfile.read(content_length)
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Connection', 'close')
        self.end_headers()
        self.wfile.write(b'{"status": "ok"}')

    def authenticate(self):
        auth_header = self.headers.get('Authorization')
        if auth_header is None:
            self.send_auth_request()
            return False

        try:
            auth_type, credentials = auth_header.split(' ', 1)
```

```
            if auth_type.lower() != 'basic':
                self.send_auth_request()
                return False

            decoded = base64.b64decode(credentials).decode('utf-8')
            username, password = decoded.split(':', 1)

            if username == USERNAME and password == PASSWORD:
                return True
        except Exception:
            pass

        self.send_auth_request()
        return False

    def send_auth_request(self):
        self.send_response(401)
        self.send_header('WWW-Authenticate', 'Basic realm="VCF Depot"')
        self.send_header('Content-type', 'text/html')
        self.send_header('Content-Length', '23')
        self.send_header('Connection', 'close')
        self.end_headers()
        self.wfile.write(b'Authentication required')

    def log_message(self, format, *args):
        print(f"{self.client_address[0]} - {format % args}")

class ThreadedHTTPServer(socketserver.ThreadingMixIn,
http.server.HTTPServer):
    daemon_threads = True

def run_server():
    os.chdir(os.path.dirname(os.path.abspath(__file__)))

    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    context.maximum_version = ssl.TLSVersion.TLSv1_3

    if hasattr(context, 'post_handshake_auth'):
        context.post_handshake_auth = False

    context.options |= ssl.OP_NO_TICKET
    context.options |= getattr(ssl, 'OP_NO_RENEGOTIATION', 0)

    context.load_cert_chain(CERT_FILE, KEY_FILE)
```

```
    try:
        context.set_ciphers('DEFAULT:!aNULL:!MD5:!DSS')
    except ssl.SSLError:
        pass

    handler = partial(AuthHandler, directory=os.getcwd())
    server = ThreadedHTTPServer(('0.0.0.0', PORT), handler)
    server.socket = context.wrap_socket(server.socket, server_side=True)

    print(f"VCF Offline Depot Server")
    print(f"========================")
    print(f"Serving: {os.getcwd()}")
    print(f"URL: https://192.168.1.160:{PORT}/")
    print(f"Credentials: {USERNAME} / {PASSWORD}")
    print(f"TLS: 1.2 - 1.3")
    print(f"Press Ctrl+C to stop")

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nStopped.")
        server.shutdown()

if __name__ == '__main__':
    run_server()
```

3.3 Certificate Generation

Certificate Generation Script (C:\VCF-DEPOT\generate_cert.py)

```
"""
Generate simple self-signed certificate for VCF Offline Depot
"""
import subprocess
import sys
import os

def generate_cert():
    try:
        from cryptography import x509
        from cryptography.x509.oid import NameOID
        from cryptography.hazmat.primitives import hashes
        from cryptography.hazmat.primitives.asymmetric import rsa
```

```
from cryptography.hazmat.primitives import serialization
from datetime import datetime, timedelta, timezone
import ipaddress

print("Generating RSA 2048-bit private key...")
key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)

print("Creating self-signed certificate...")
subject = issuer = x509.Name([
    x509.NameAttribute(NameOID.COMMON_NAME, "192.168.1.160"),
])

now = datetime.now(timezone.utc)

# Simple certificate like the original
cert = (
    x509.CertificateBuilder()
    .subject_name(subject)
    .issuer_name(issuer)
    .public_key(key.public_key())
    .serial_number(x509.random_serial_number())
    .not_valid_before(now)
    .not_valid_after(now + timedelta(days=365))
    .add_extension(
        x509.SubjectAlternativeName([
            x509.IPAddress(ipaddress.IPv4Address("192.168.1.160")),
        ]),
        critical=False,
    )
    .add_extension(
        x509.BasicConstraints(ca=True, path_length=None),
        critical=True,
    )
    .sign(key, hashes.SHA256())
)

with open("server.key", "wb") as f:
    f.write(key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption()
    ))
print("Created: server.key")
```

```
        with open("server.crt", "wb") as f:
            f.write(cert.public_bytes(serialization.Encoding.PEM))
        print("Created: server.crt")

        fingerprint = cert.fingerprint(hashes.SHA256()).hex()
        formatted = ':'.join(fingerprint[i:i+2].upper() for i in range(0,
len(fingerprint), 2))
        print(f"SHA256: {formatted}")

        return True
    except ImportError:
        print("Installing cryptography...")
        subprocess.check_call([sys.executable, "-m", "pip", "install",
"cryptography"])
        return False

def main():
    os.chdir(r"C:\VCF-DEPOT")
    print("Generating certificate...")
    if not generate_cert():
        generate_cert()
    print("\nDone. Run: python https_server.py")

if __name__ == "__main__":
    main()
```

Windows PowerShell Commands

```
# Navigate to depot directory
cd C:\VCF-DEPOT

# Generate certificate
python generate_cert.py

# Start HTTPS server
python https_server.py
```

3.4 Certificate Import to VCF

Complete Certificate Import Procedure (VCF Installer)

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Step 1: Download certificate from depot server
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl
x509 -outform PEM > /tmp/depot.crt

# Step 2: Verify certificate was downloaded
cat /tmp/depot.crt

# Step 3: Get certificate fingerprint
openssl x509 -in /tmp/depot.crt -noout -fingerprint -sha256

# Step 4: Find Java truststore location
echo $JAVA_HOME
# Output: /usr/lib/jvm/openjdk-java17-headless.x86_64

# Step 5: Delete old certificate if exists
keytool -delete -alias offline-depot -keystore
$JAVA_HOME/lib/security/cacerts -storepass changeit

# Step 6: Import new certificate
keytool -import -trustcacerts -alias offline-depot -file /tmp/depot.crt -
keystore $JAVA_HOME/lib/security/cacerts -storepass changeit -noprompt

# Step 7: Verify import
keytool -list -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts
-storepass changeit

# Step 8: Restart LCM service
systemctl restart lcm

# Step 9: Wait for LCM to start (2 minutes)
systemctl status lcm

# Step 10: Verify LCM is ready
tail -f /var/log/vmware/vcf/lcm/lcm-debug.log | grep -i "started\|ready"
```

Appliance-only: *`keytool`, `systemctl`, file ops under `/tmp` and `/var/log` all run inside the VCF Installer appliance — no PowerShell sibling. SSH in as `vcf`, then `su -` to root.*

Troubleshooting Certificate Issues

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Check all cacerts files on system
find / -name "cacerts" -type f 2>/dev/null

# List all certificates in truststore
keytool -list -keystore $JAVA_HOME/lib/security/cacerts -storepass changeit

# View certificate details in truststore
keytool -list -alias offline-depot -keystore $JAVA_HOME/lib/security/cacerts
-storepass changeit -v

# Check LCM logs for certificate errors
grep -B5 -A10 "TlsFatalAlert" /var/log/vmware/vcf/lcm/lcm-debug.log | tail
-40
```

Appliance-only: *`keytool` and log inspection under `/var/log/vmware/vcf/lcm/` run inside the VCF Installer appliance — no PowerShell sibling.*

4. ESXi Host Certificate Issues

4.1 Certificate Hostname Mismatch

Problem Description

ESXi hosts may have certificates with incorrect hostnames (e.g., "localhost.localdomain" instead of the actual FQDN), causing VCF validation to fail.

Symptoms

```
javax.net.ssl.SSLPeerUnverifiedException: Certificate for <esxi01.lab.local>
doesn't match any of the subject alternative names: [localhost.localdomain]
```

Diagnostic Commands (ESXi Host)

```
# bash on ESXi host via SSH (appliance-only)
# Check current hostname
esxcli system hostname get

# View current certificate SAN
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"

# View full certificate details
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout
```

Appliance-only: *`esxcli` and `/etc/vmware/ssl/` inspection run on the ESXi host shell — no PowerShell sibling. SSH in as `root` and run there. (PowerCLI `Get-VMHost` / `Get-EsxCli` could be used to wrap a subset of `esxcli` calls remotely — left out here because cert-file inspection requires shell access.)*

4.2 Certificate Regeneration

Procedure (Run on Each ESXi Host)

```
# bash on ESXi host via SSH (appliance-only)
# Step 1: Set correct hostname
esxcli system hostname set --fqdn=esxi01.lab.local

# Step 2: Verify hostname
esxcli system hostname get

# Step 3: Backup existing certificate
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak

# Step 4: Generate new certificate
/sbin/generate-certificates

# Step 5: Restart services
services.sh restart

# Step 6: Verify new certificate
```

```
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"
```

> ***Appliance-only:*** *`esxcli`, `/sbin/generate-certificates`, `services.sh`, and on-host file moves under `/etc/vmware/ssl/` all run on the ESXi host shell — no PowerShell sibling.*

Host-Specific Commands

esxi01.lab.local (192.168.1.74):

```
# bash on ESXi host via SSH (appliance-only)
esxcli system hostname set --fqdn=esxi01.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

esxi02.lab.local (192.168.1.75):

```
# bash on ESXi host via SSH (appliance-only)
esxcli system hostname set --fqdn=esxi02.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

esxi03.lab.local (192.168.1.76):

```
# bash on ESXi host via SSH (appliance-only)
esxcli system hostname set --fqdn=esxi03.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

esxi04.lab.local (192.168.1.82):

```
# bash on ESXi host via SSH (appliance-only)
esxcli system hostname set --fqdn=esxi04.lab.local
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
```

Appliance-only: *All four host-specific blocks run on the ESXi host shells — no PowerShell sibling.*

4.3 Thumbprint Updates

Get New Thumbprints (VCF Installer)

```
# bash on VCF Installer (or any Linux host with openssl)
# Get thumbprint for each host
echo | openssl s_client -connect 192.168.1.74:443 2>/dev/null | openssl x509
-noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.75:443 2>/dev/null | openssl x509
-noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.76:443 2>/dev/null | openssl x509
-noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.82:443 2>/dev/null | openssl x509
-noout -fingerprint -sha256
```

```
# PowerShell on Windows workstation -- pull SHA256 thumbprint per ESXi host
$Hosts = @(
    @{ Name = 'esxi01.lab.local'; Address = '192.168.1.74' }
    @{ Name = 'esxi02.lab.local'; Address = '192.168.1.75' }
    @{ Name = 'esxi03.lab.local'; Address = '192.168.1.76' }
    @{ Name = 'esxi04.lab.local'; Address = '192.168.1.82' }
)

[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {
$true }

foreach ($h in $Hosts) {
    $tcp = New-Object System.Net.Sockets.TcpClient($h.Address, 443)
```

```
    $ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
({ $true }))
    $ssl.AuthenticateAsClient($h.Address)
    $cert = New-Object
System.Security.Cryptography.X509Certificates.X509Certificate2($ssl.RemoteCerti1
    $sha  = (Get-FileHash -InputStream
([IO.MemoryStream]::new($cert.RawData)) -Algorithm SHA256).Hash
    "{0,-22} {1}" -f $h.Name, ($sha -replace '(.{2})(?!$)','$1:')
    $ssl.Dispose(); $tcp.Close()
}
```

After regenerating certificates, update the thumbprints in VCF Installer UI by re-validating the hosts.

5. vSAN Storage Configuration

5.1 SSD Detection Issues

Problem Description

In nested VMware Workstation environments, virtual disks are not automatically detected as SSDs, causing vSAN cache tier configuration to fail.

Symptoms

```
ESX Host esxi01.lab.local found zero SSD devices for SSD cache tier
```

Diagnostic Commands (ESXi Host)

```
# bash on ESXi host via SSH (appliance-only)
# List all storage devices with SSD status
esxcli storage core device list | grep -E "^t10|Is SSD"

# Check vSAN eligible disks
vdq -q

# List vSAN storage
esxcli vsan storage list
```

```
# Check disk partitions
partedUtil getptbl /vmfs/devices/disks/<device-name>
```

Appliance-only: *`esxcli`, `vdq`, and `partedUtil` run inside the ESXi shell — no PowerShell sibling. Some `esxcli` views can be wrapped via PowerCLI `Get-EsxCli -V2`, but `vdq` / `partedUtil` are shell-only.*

5.2 VMX File Configuration

Problem Description

Virtual disks in VMware Workstation need the `virtualSSD` flag set in the VMX file to be recognized as SSDs by nested ESXi.

Solution: Edit VMX Files

Location: Edit each ESXi VM's .vmx file in VMware Workstation

Required Lines to Add:

For **esxi01.vmx**:

```
sata0:0.virtualSSD = 1
sata0:2.virtualSSD = 1
sata0:3.virtualSSD = 1
sata0:4.virtualSSD = 1
```

For **esxi02.vmx**, **esxi03.vmx**, **esxi04.vmx**:

```
sata0:0.virtualSSD = 1
sata0:3.virtualSSD = 1
sata0:4.virtualSSD = 1
```

Procedure

1. Shut down all ESXi VMs in VMware Workstation
2. Navigate to each VM's folder
3. Open the .vmx file in a text editor
4. Add the virtualSSD lines (location doesn't matter, bottom is fine)

5. Save the file
6. Power on the ESXi VMs
7. Verify SSD detection (appliance-only — ESXi shell):

```
# bash on ESXi host via SSH (appliance-only)
esxcli storage core device list | grep -E "^t10|Is SSD"
```

5.3 Cleaning Up Previous vSAN Configuration

Problem Description

Disks with existing vSAN partitions from previous deployments are marked as "Ineligible for use by VSAN" with reason "Has partitions" or "Disk in use by disk group".

Diagnostic Commands

```
# bash on ESXi host via SSH (appliance-only)
# Check vSAN eligibility
vdq -q

# Check existing vSAN storage
esxcli vsan storage list

# Check partition table
partedUtil getptbl
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
```

Appliance-only: *`vdq`, `esxcli`, `partedUtil` run inside the ESXi shell — no PowerShell sibling.*

Cleanup Procedure (Run on Each ESXi Host)

```
# bash on ESXi host via SSH (appliance-only)
# Step 1: Remove existing vSAN disk group (if exists)
esxcli vsan storage remove -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001
```

```
# Step 2: Delete partitions from vSAN cache disk
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
2

# Step 3: Delete partitions from vSAN capacity disk
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000
2

# Step 4: Verify disks are now eligible
vdq -q
```

Appliance-only: *`esxcli vsan storage remove`, `partedUtil`, and `vdq` run inside the ESXi shell — no PowerShell sibling.*

Expected Output After Cleanup


```
{
    "Name":
"t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001",
    "State": "Eligible for use by VSAN",
    "Reason": "None",
    "IsSSD": "1"
}
```


6. Network Configuration for Nested VCF

6.1 VLAN Configuration

Problem Description

VCF requires separate subnets for different network types. Using the same subnet/gateway for multiple networks causes validation errors:

```
Gateway 192.168.1.1 is duplicated across networks
Subnet 192.168.1.0/24 is duplicated across networks
```

Solution: Use VLANs with Separate Subnets

Network	VLAN ID	Subnet	Gateway
Management	0	192.168.1.0/24	192.168.1.1
vMotion	100	192.168.100.0/24	192.168.100.1
vSAN	200	192.168.200.0/24	192.168.200.1
NSX TEP	300	192.168.250.0/24	192.168.250.1

Note: For vMotion, vSAN, and TEP networks in nested environments, the gateway IPs don't need to exist - they're isolated networks. VCF requires the gateway field to be populated.

6.2 MTU Settings

Critical for Nested Environments

Do NOT use jumbo frames (MTU 9000) in nested VMware Workstation environments.

Component	Recommended MTU
Distributed Switch	1500-1600
ESX Management	1500
vMotion	1500
vSAN	1500
NSX TEP	1500

6.3 IP Addressing

VCF Network IP Ranges

Network	IP Range	Purpose
Management	192.168.1.x	Static assignments per hosts file
vMotion	192.168.100.10-20	Automatic assignment by VCF
vSAN	192.168.200.206-216	Automatic assignment by VCF
NSX TEP	192.168.250.10-25	TEP IP Pool

7. VCF Component Configuration

7.1 VCF Automation

IP Address Configuration Issue

Problem: VCF detects duplicate IP when cluster FQDN resolves to same IP as Node IP.

Error:

```
IP address 192.168.1.90 for product VCF Automation is already resolved from an FQDN in the input specification
```

Solution: Use different IPs for cluster FQDN and node IP:

Field	Value
Cluster hostname/FQDN	automation.lab.local (192.168.1.90)
Node IP 1	192.168.1.91 (automation-node1.lab.local)
Additional IP for upgrades	192.168.1.81
Node name prefix	automation
Internal Cluster CIDR	198.18.0.0/15

7.2 NSX Manager

Configuration

Field	Value
Appliance Size	Medium
Appliance FQDN	nsx-node1.lab.local
Virtual IP (VIP) FQDN	nsx-vip.lab.local

7.2.1 NSX Certificate SAN Failure (VDT)

Problem: VDT reports "SAN contains neither hostname nor IP" for NSX VIP and NSX Manager certificates. The default NSX self-signed certificate uses a wildcard SAN (`*.lab.local`) without specific hostnames or IPs.

Solution: Generate a new self-signed certificate with explicit SAN entries and apply via NSX API.

Step 1: Create OpenSSL config on NSX Manager (SSH as root):

```
# bash on NSX Manager appliance via SSH (appliance-only)
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
```

```
IP.2 = 192.168.1.71
EOF
```

> ***Appliance-only:*** *Writing to* `/tmp` *and subsequent* `openssl req` *happen inside the NSX Manager appliance shell — no PowerShell sibling for the file write itself. The cert can be* ***applied*** *via NSX REST API from the workstation (see Step 2 sibling below).*

Important: `DNS.3 = nsx-manager.lab.local` is required because SDDC Manager registers NSX using this FQDN. Without it, VDT reports "SAN contains IP but not hostname".

Step 2: Generate cert, build JSON, import, and apply:

```
# bash on NSX Manager appliance via SSH (appliance-only for openssl/python;
API calls workstation-runnable)
# Generate cert
openssl req -x509 -nodes -days 825 -newkey rsa:2048 -keyout /tmp/nsx.key -out
/tmp/nsx.crt -config /tmp/nsx-cert.conf -sha256

# Build JSON payload (avoids shell escaping issues with PEM newlines)
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json

# Import cert (single-line — NSX shell doesn't support backslash
continuation)
curl -k -u admin:'<password>' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates?action=import" -H "Content-Type: application/json" -d
@/tmp/nsx-import.json
# Note the certificate ID from the response

# Get node UUID
curl -k -u admin:'<password>' https://192.168.1.71/api/v1/cluster
# Note the node UUID

# Apply to node (API service)
curl -k -u admin:'<password>' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=API&node_id=<node-uuid>"
```

```
# Apply to VIP (MGMT_CLUSTER)
curl -k -u admin:'<password>' -X POST "https://192.168.1.71/api/v1/trust-
management/certificates/<cert-id>?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

```
# PowerShell on Windows workstation -- import + apply NSX cert via REST
# (Cert/key PEMs must already exist on the workstation, e.g.
C:\certs\nsx.crt, C:\certs\nsx.key)
$NsxNode  = '192.168.1.71'
$NsxUser  = 'admin'
$NsxPass  = '<password>'   # consider Get-Credential for production
$CertPath = 'C:\certs\nsx.crt'
$KeyPath  = 'C:\certs\nsx.key'

# Cert-bypass for self-signed NSX
[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]'Tls12,Tls13'
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {
$true }

$pair = "$($NsxUser):$($NsxPass)"
$auth = 'Basic ' +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($pair))
$hdrs = @{ Authorization = $auth; 'Content-Type' = 'application/json' }

# Build JSON payload from PEM files (preserves newlines via JSON serializer)
$body = @{
    pem_encoded = (Get-Content -Raw $CertPath)
    private_key = (Get-Content -Raw $KeyPath)
} | ConvertTo-Json -Depth 4

# Import cert
$import = Invoke-RestMethod -Method Post `
    -Uri "https://$NsxNode/api/v1/trust-management/certificates?
action=import" `
    -Headers $hdrs -Body $body
$CertId = $import.results[0].id
"Imported cert id: $CertId"

# Get node UUID
$cluster = Invoke-RestMethod -Method Get `
    -Uri "https://$NsxNode/api/v1/cluster" -Headers $hdrs
$NodeId = $cluster.nodes[0].node_uuid
"Node UUID: $NodeId"
```

```
# Apply to node (API service)
Invoke-RestMethod -Method Post -Headers $hdrs `
    -Uri "https://$NsxNode/api/v1/trust-management/certificates/$CertId`?
action=apply_certificate&service_type=API&node_id=$NodeId"

# Apply to VIP (MGMT_CLUSTER)
Invoke-RestMethod -Method Post -Headers $hdrs `
    -Uri "https://$NsxNode/api/v1/trust-management/certificates/$CertId`?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Prerequisite: All NSX services must be healthy (MANAGER, SEARCH, UI, NODE_MGMT all UP). If services are DOWN, the API returns error 101. Wait 10-15 minutes after NSX restart in nested environments.

7.2.2 NSX Certificate Trust Failure (VDT)

Problem: After replacing the NSX self-signed certificate, VDT reports "NSX VIP Cert Trust: FAIL" and "NSX Manager Cert Trust: FAIL". The new self-signed cert's root is not in SDDC Manager's keystores (the original cert was pre-trusted during bringup).

Symptoms:

- VDT shows FAIL for NSX VIP Cert Trust and NSX Manager Cert Trust
- NSX cert SAN checks may PASS but trust checks FAIL
- Occurs after any NSX certificate replacement with a new self-signed cert

Solution: Import the NSX certificate into both SDDC Manager trust stores.

Step 1: Pull the NSX certificate (SSH to SDDC Manager as root):

```
# bash on SDDC Manager appliance via SSH (appliance-only)
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null
| openssl x509 -outform PEM > /tmp/nsx-root.crt

# Verify it's the correct cert
openssl x509 -in /tmp/nsx-root.crt -noout -text | grep -A2 "Subject
Alternative Name"
```

> ***Appliance-only:*** *Pulling the NSX cert and writing it under* `/tmp/` *happens on the SDDC Manager appliance itself so the keytool import in Step 2/3 can read it — no PowerShell sibling.*

Step 2: Import into VCF trust store:

```
# bash on SDDC Manager appliance via SSH (appliance-only)
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Appliance-only: *`keytool` against `/etc/vmware/vcf/commonsvcs/` runs on the SDDC Manager appliance — no PowerShell sibling.*

Step 3: Import into Java cacerts:

```
# bash on SDDC Manager appliance via SSH (appliance-only)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt
```

Appliance-only: *`keytool` against `/etc/alternatives/jre/lib/security/cacerts` runs on the SDDC Manager appliance — no PowerShell sibling.*

Step 4: Restart SDDC Manager services (~5 minutes):

```
# bash on SDDC Manager appliance via SSH (appliance-only)
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

Appliance-only: *The restart script lives under `/opt/vmware/vcf/operationsmanager/` on SDDC Manager — no PowerShell sibling.*

Key paths:

Item	Path/Value
VCF trust store	`/etc/vmware/vcf/commonsvcs/trusted_certificates.store`

VCF trust store password	Contents of `/etc/vmware/vcf/commonsvcs/trusted_certificates.key`
Java cacerts	`/etc/alternatives/jre/lib/security/cacerts`
Java cacerts password	`changeit`
Service restart script	`/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh`

Reference: https://knowledge.broadcom.com/external/article/316056

Note on SDDC Manager SSH: Only the `vcf` user can SSH in (root and admin are rejected). Use `su -` from the vcf session to get root access. SCP does not work due to the restricted shell; use `ssh vcf@host "cat > file" < localfile` for file transfers.

7.3 vCenter Server

Configuration

Field	Value
Appliance FQDN	vcenter.lab.local
Appliance Size	Tiny
Datacenter Name	mgmt-dc01
Cluster Name	mgmt-cl01
SSO Domain Name	vsphere.local

7.4 vCenter Deployment Issues

7.4.1 SSH Requirement for vLCM Host Seeding

Problem Description VCF vLCM (vSphere Lifecycle Manager) requires SSH access to ESXi hosts during vCenter deployment for host seeding. If SSH is disabled, the deployment fails with:

```
vCenter installation failed. Check logs under
/var/log/vmware/vcf/domainmanager/ci-installer-XX-XX-XX-XX-XX-XXX for more
details
```

Symptoms in Logs

```
Extraction of image from host esxi01.lab.local failed
```

Root Cause SSH service is stopped or disabled on ESXi hosts. VCF needs SSH to extract ESXi image metadata for vLCM host seeding.

Solution: Enable SSH on All ESXi Hosts

Run on each ESXi host BEFORE starting VCF deployment:

```
# bash on ESXi host shell (DCUI/console) -- BEFORE SSH is enabled
# Enable SSH service
vim-cmd hostsvc/enable_ssh

# Start SSH service
vim-cmd hostsvc/start_ssh

# Verify SSH is running
vim-cmd hostsvc/runtimeinfo | grep ssh
```

```
# PowerShell on Windows workstation via PowerCLI
# Enable + start SSH on every host in a cluster (or list specific hosts)
Connect-VIServer -Server vcenter.lab.local -User
'administrator@vsphere.local' -Password '<password>'

$Hosts = Get-VMHost                                    # or Get-VMHost
esxi01.lab.local, esxi02.lab.local
foreach ($vmh in $Hosts) {
    $svc = Get-VMHostService -VMHost $vmh | Where-Object { $_.Key -eq 'TSM-
SSH' }
    if ($svc.Running) { "[{0}] SSH already running" -f $vmh.Name }
    else {
        Start-VMHostService -HostService $svc -Confirm:$false | Out-Null
        "[{0}] SSH started" -f $vmh.Name
    }
```

```
}

Disconnect-VIServer -Server vcenter.lab.local -Confirm:$false
```

Note: *The bash* `vim-cmd` *form runs on the ESXi DCUI/console* ***before*** *SSH is up, so the PowerCLI sibling is the recommended workstation path* ***after*** *vCenter is reachable. If vCenter is not yet deployed (which is the entire reason this section exists), you must use the DCUI/console for* `vim-cmd`*.*

Alternative Method (from ESXi Shell)

```
# bash on ESXi host via SSH (appliance-only)
# Enable and start SSH
esxcli system ssh set --enable=true

# Verify SSH status
esxcli system ssh get
```

Appliance-only: *Direct* `esxcli` *runs on the ESXi shell. PowerCLI sibling above already covers the workstation path via vCenter; if vCenter is not yet up, no remote PowerShell is possible — use the host DCUI.*

Note: SSH can be disabled after successful VCF deployment for security.

7.4.2 Monitoring vCenter Deployment Progress

VCF Installer Log Monitoring

Watch deployment progress from VCF Installer:

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Find the latest ci-installer log directory
ls -lt /var/log/vmware/vcf/domainmanager/ | head -5

# Watch the installation log
tail -f /var/log/vmware/vcf/domainmanager/ci-installer-XX-XX-XX-XX-XX-XXX/ci-
installer.log

# Search for errors
```

```
grep -i "error\|failed\|exception" /var/log/vmware/vcf/domainmanager/ci-installer-XX-XX-XX-XX-XX-XXX/ci-installer.log
```

Appliance-only: *Log inspection under `/var/log/vmware/vcf/domainmanager/` runs on the VCF Installer appliance — no PowerShell sibling.*

vCenter VM Direct Monitoring

SSH to the vCenter VM during deployment (default password: vmware):

```
# bash on vCenter VM via SSH (appliance-only)
# Watch firstboot progress
tail -f /var/log/firstboot/firstbootStatus.json

# Watch detailed installation
tail -f /var/log/vmware/firstboot/installer.log

# Check VMware services
vmon-cli --list

# Check specific service status
vmon-cli --status <service-name>
```

Appliance-only: *`vmon-cli` and firstboot log files live inside the vCenter appliance — no PowerShell sibling.*

Expected Deployment Stages

1. vCenter VM deployment (OVA extraction)
2. First boot - basic configuration
3. Installing containers (60% mark)
4. Database initialization
5. Service startup
6. vCenter registration with VCF

7.4.3 Diagnosing Stuck Deployments

Symptoms

- Deployment stuck at a percentage (commonly 60%) for more than 30 minutes
- No progress in firstboot logs
- VCF UI shows no timeout error

Diagnostic Commands (SSH to vCenter VM)

```
# bash on vCenter VM via SSH (appliance-only)
# Check current deployment status
cat /var/log/firstboot/firstbootStatus.json

# Check for running processes
ps aux | grep -E "install|firstboot|postgres|vpxd"

# Check disk I/O (should show activity)
vmstat 1 5

# Check memory usage
free -h

# Check for error logs
tail -50 /var/log/vmware/firstboot/installer.log
grep -i "error\|fail\|exception" /var/log/vmware/firstboot/*.log
```

> ***Appliance-only:*** *Process inspection,* `vmstat` *,* `free` *, and firstboot logs all live inside the vCenter appliance — no PowerShell sibling.*

PostgreSQL Database Issues

If deployment is stuck at "Installing Containers" (60%), check postgres:

```
# bash on vCenter VM via SSH (appliance-only)
# Check if postgres service exists
ls -la /storage/db/vpostgres/

# Check for postgres config file
ls -la /storage/db/vpostgres/postgresql.conf

# Check postgres user/group
grep postgres /etc/passwd
grep postgres /etc/group
```

```
# Check postgres logs
tail -50 /var/log/vmware/vpostgres/*.log
```

Appliance-only: *`/storage/db/vpostgres/`, `/etc/passwd`, and `/var/log/vmware/vpostgres/` all live inside the vCenter appliance — no PowerShell sibling.*

If PostgreSQL Never Initialized Missing `/storage/db/vpostgres/postgresql.conf` and missing postgres user indicates the database initialization failed. This is typically unrecoverable and requires full redeployment.

Service Startup Issues

```
# bash on vCenter VM via SSH (appliance-only)
# List all VMware services and status
vmon-cli --list

# Check rhttpproxy (reverse proxy)
systemctl status rhttpproxy
tail -50 /var/log/vmware/rhttpproxy/rhttpproxy.log

# Check vpostgres
systemctl status vmware-vpostgres
tail -50 /var/log/vmware/vpostgres/postgresql*.log
```

Appliance-only: *`vmon-cli`, `systemctl`, and `/var/log/vmware/` access run inside the vCenter appliance — no PowerShell sibling.*

7.4.4 vCenter Deployment Failure Reference Tokens

When vCenter deployment fails, VCF provides a reference token. To find detailed errors:

```
# bash on VCF Installer / SDDC Manager appliance via SSH (appliance-only)
# Search for reference token in logs
grep -r "REFERENCE_TOKEN" /var/log/vmware/vcf/

# Example: Reference Token 3OHCKD
```

```
grep -r "3OHCKD" /var/log/vmware/vcf/
grep -B20 -A20 "3OHCKD" /var/log/vmware/vcf/domainmanager/*.log
```

Appliance-only: *Searching `/var/log/vmware/vcf/` requires SSH access to the VCF Installer / SDDC Manager appliance — no PowerShell sibling.*

7.5 Failed Deployment Recovery

7.5.1 Overview

VCF does not provide a rollback mechanism for failed management domain deployments. A failed deployment requires manual cleanup of:

1. vCenter VM (delete from ESXi)
2. vSAN disk groups and partitions
3. VDS (Virtual Distributed Switch) configuration
4. Depot connection in VCF UI

7.5.2 Complete Cleanup Procedure

Step 1: Delete Failed vCenter VM

From any ESXi host (or the one hosting the failed vCenter):

```
# bash on ESXi host via SSH (appliance-only)
# List all VMs
vim-cmd vmsvc/getallvms

# Find the vCenter VM ID (look for vcenter.lab.local or similar)
# Power off the VM if running
vim-cmd vmsvc/power.off <vmid>

# Unregister the VM
vim-cmd vmsvc/unregister <vmid>

# Delete the VM files from datastore (if needed)
rm -rf /vmfs/volumes/<datastore>/vcenter.lab.local/
```

> ***Appliance-only:*** *`vim-cmd` and direct datastore `rm` against `/vmfs/volumes/` run on the ESXi host shell — no PowerShell sibling. (PowerCLI `Remove-VM -DeletePermanently` requires vCenter, which is precisely the VM being deleted here.)*

Step 2: Clean Up VDS (Distributed Switch)

From ESXi hosts, remove VDS configuration:

```
# bash on ESXi host via SSH (appliance-only)
# List current virtual switches
esxcli network vswitch dvs vmware list

# Remove host from VDS (if configured)
# This is typically done from vCenter, but if vCenter is gone:
# Remove vmkernel ports from VDS
esxcli network ip interface remove -i vmk1  # vMotion
esxcli network ip interface remove -i vmk2  # vSAN

# Remove VDS uplink
esxcli network vswitch dvs vmware list
```

> ***Appliance-only:*** *`esxcli network` runs on the ESXi shell — no PowerShell sibling. PowerCLI VDS cmdlets all require vCenter, which is gone in this recovery path.*

Step 3: Clean Up vSAN Configuration

Run on EACH ESXi host:

```
# bash on ESXi host via SSH (appliance-only)
# List current vSAN storage
esxcli vsan storage list

# Remove vSAN disk groups
esxcli vsan storage remove -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000000000000001
esxcli vsan storage remove -d
t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000000000000001

# If remove fails, check disk state
```

```
vdq -q

# Delete partitions from cache disk (example device name)
partedUtil getptbl
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________03000000
2

# Delete partitions from capacity disk
partedUtil getptbl
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000
1
partedUtil delete
/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive__________04000000
2

# Verify disks are now eligible
vdq -q
```

> ***Appliance-only:*** *`esxcli vsan storage`, `vdq`, and `partedUtil` all run on the ESXi shell — no PowerShell sibling.*

Common vSAN Cleanup Error If you see: `cache disk/s are in an invalid state...available size is 0.0 GB`

This means disks still have partitions. Use partedUtil to delete them.

Step 4: Remove Depot Connection (VCF UI)

1. Log into VCF Installer UI (https://192.168.1.240:8443)
2. Navigate to Settings or Configuration
3. Remove the existing offline depot connection
4. Re-add the depot connection with certificate

Step 5: Verify Hosts Are Ready

On each ESXi host, verify:

```
# bash on ESXi host via SSH (appliance-only)
# Check hostname is correct
esxcli system hostname get

# Check certificate is valid
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"

# Check SSH is enabled
vim-cmd hostsvc/runtimeinfo | grep ssh

# Check vSAN disks are eligible
vdq -q

# Check no VDS remnants
esxcli network vswitch dvs vmware list
```

> ***Appliance-only:*** *All five checks above run on the ESXi shell — no PowerShell sibling.*

Step 6: Restart VCF Services (Optional)

On VCF Installer, restart services for clean state:

```
# bash on VCF Installer appliance via SSH (appliance-only)
systemctl restart lcm
systemctl restart domainmanager

# Wait for services to start
sleep 120

# Verify services are running
systemctl status lcm
systemctl status domainmanager
```

> ***Appliance-only:*** `systemctl` *runs inside the VCF Installer appliance — no PowerShell sibling.*

7.5.3 Troubleshooting Flowchart: Failed Deployment Recovery

```
START: VCF Deployment Failed
  │
  ├─→ Note reference token from error message
  │     └─→ Search logs: grep -r "TOKEN" /var/log/vmware/vcf/
  │
  ├─→ Delete failed vCenter VM
  │     ├─→ vim-cmd vmsvc/getallvms
  │     ├─→ vim-cmd vmsvc/power.off <vmid>
  │     └─→ vim-cmd vmsvc/unregister <vmid>
  │
  ├─→ Clean up vSAN on EACH host
  │     ├─→ esxcli vsan storage remove -d <device>
  │     ├─→ partedUtil delete ... (both partitions)
  │     └─→ vdq -q (verify eligible)
  │
  ├─→ Clean up VDS (if configured)
  │     └─→ esxcli network ip interface remove ...
  │
  ├─→ Remove depot connection in VCF UI
  │     └─→ Re-add with certificate
  │
  ├─→ Verify SSH enabled on all hosts
  │     └─→ vim-cmd hostsvc/enable_ssh
  │
  └─→ Retry deployment
```

8. Command Reference

8.1 VCF Installer Commands

Service Management

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Check LCM service status
systemctl status lcm

# Restart LCM service
systemctl restart lcm
```

```
# Check Domain Manager status
systemctl status domainmanager

# Restart Domain Manager
systemctl restart domainmanager

# View LCM logs
tail -f /var/log/vmware/vcf/lcm/lcm-debug.log

# Search LCM logs for errors
grep -i "error\|fatal\|exception" /var/log/vmware/vcf/lcm/lcm-debug.log |
tail -20
```

Appliance-only: *`systemctl` and `/var/log/vmware/vcf/lcm/` access run inside the VCF Installer appliance — no PowerShell sibling.*

Certificate Management

```
# bash on VCF Installer appliance via SSH (appliance-only)
# Download certificate from remote server
openssl s_client -connect <IP>:<PORT> </dev/null 2>/dev/null | openssl x509 -
outform PEM > /tmp/cert.crt

# View certificate details
openssl x509 -in /tmp/cert.crt -text -noout

# Get certificate fingerprint
openssl x509 -in /tmp/cert.crt -noout -fingerprint -sha256

# Import certificate to Java truststore
keytool -import -trustcacerts -alias <alias> -file /tmp/cert.crt -keystore
$JAVA_HOME/lib/security/cacerts -storepass changeit -noprompt

# Delete certificate from truststore
keytool -delete -alias <alias> -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit

# List certificates in truststore
keytool -list -keystore $JAVA_HOME/lib/security/cacerts -storepass changeit

# View specific certificate in truststore
```

```
keytool -list -alias <alias> -keystore $JAVA_HOME/lib/security/cacerts -
storepass changeit -v
```

Appliance-only: *`keytool` against the appliance Java cacerts and writing to `/tmp` happens on the VCF Installer / SDDC Manager appliance — no PowerShell sibling. The certificate-fingerprint* ***download*** *step has a workstation-side equivalent shown in section 4.3 above.*

Network Diagnostics

```
# bash on workstation OR appliance
# Test connectivity
ping <IP>

# Test SSL connection
openssl s_client -connect <IP>:<PORT>

# Test with specific TLS version
openssl s_client -connect <IP>:<PORT> -tls1_2

# Test HTTP endpoint
curl -v -k -u admin:admin https://<IP>:<PORT>/path
```

```
# PowerShell on Windows workstation -- network diagnostics
$Target = '<IP>'
$Port   = '<PORT>'
$User   = 'admin'
$Pass   = 'admin'

# Connectivity (ICMP)
Test-Connection -ComputerName $Target -Count 4

# TCP/TLS reachability + version
Test-NetConnection -ComputerName $Target -Port $Port -InformationLevel
Detailed

# TLS handshake -- pull protocol + cert
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {
$true }
$tcp = New-Object System.Net.Sockets.TcpClient($Target, $Port)
```

```
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({
$true }))
$ssl.AuthenticateAsClient($Target)
"Negotiated: $($ssl.SslProtocol) / $($ssl.CipherAlgorithm)"
$ssl.Dispose(); $tcp.Close()

# HTTP endpoint test (verbose-ish)
[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]'Tls12,Tls13'
$pair = "$($User):$($Pass)"
$auth = 'Basic ' +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($pair))
Invoke-WebRequest -Uri "https://$Target`:$Port/path" `
    -Headers @{ Authorization = $auth } -SkipCertificateCheck | Select-Object
StatusCode, StatusDescription, Headers
```

Note: *`-SkipCertificateCheck` requires PowerShell 7+. On PS 5.1, rely on the global `ServerCertificateValidationCallback` shim shown above.*

8.2 ESXi Host Commands

System Information

```
# bash on ESXi host via SSH (appliance-only)
# Get hostname
esxcli system hostname get

# Set hostname
esxcli system hostname set --fqdn=<FQDN>

# Get system version
esxcli system version get
```

```
# PowerShell on Windows workstation via PowerCLI (read-only equivalents)
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -
Password '<password>'

# Get hostname / FQDN
```

```
Get-VMHost | Select-Object Name, @{N='FQDN';E=
{$_.ExtensionData.Config.Network.DnsConfig.HostName + '.' +
$_.ExtensionData.Config.Network.DnsConfig.DomainName}}

# Set FQDN -- requires Get-EsxCli (or DCUI) since it modifies host identity
$esx = (Get-EsxCli -V2 -VMHost (Get-VMHost esxi01.lab.local))
$esx.system.hostname.set.Invoke(@{ fqdn = 'esxi01.lab.local' })

# Get version
Get-VMHost | Select-Object Name, Version, Build
```

Note: *Setting the hostname via PowerCLI's `Get-EsxCli` proxy requires vCenter to be up and the host added to inventory. If vCenter does not yet exist, use the bash form on the host itself.*

Certificate Management

```
# bash on ESXi host via SSH (appliance-only)
# View current certificate
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout

# View certificate SAN
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"

# Backup certificates
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak

# Regenerate certificates
/sbin/generate-certificates

# Restart services after certificate change
services.sh restart
```

Appliance-only: *Reading and moving files under `/etc/vmware/ssl/`, running `/sbin/generate-certificates`, and `services.sh restart` all require ESXi shell access — no PowerShell sibling. (See section 4.3 for the workstation-runnable thumbprint pull.)*

Storage Management

```
# bash on ESXi host via SSH (appliance-only)
# List all storage devices
esxcli storage core device list

# List devices with SSD status
esxcli storage core device list | grep -E "^t10|^naa|Is SSD"

# Check vSAN eligible disks
vdq -q

# List vSAN storage
esxcli vsan storage list

# Get partition table
partedUtil getptbl /vmfs/devices/disks/<device>

# Delete partition
partedUtil delete /vmfs/devices/disks/<device> <partition-number>

# Remove vSAN disk group
esxcli vsan storage remove -d <device>

# Add SATP rule for SSD
esxcli storage nmp satp rule add -s VMW_SATP_LOCAL -d <device> -o enable_ssd

# Reclaim device after SATP rule
esxcli storage core claiming reclaim -d <device>

# List SATP rules
esxcli storage nmp satp rule list | grep enable_ssd
```

```
# PowerShell on Windows workstation via PowerCLI (read-only -- destructive
ops via bash on host)
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -
Password '<password>'

$VMHost = Get-VMHost esxi01.lab.local
$esx    = Get-EsxCli -V2 -VMHost $VMHost

# List all storage devices
```

```
$esx.storage.core.device.list.Invoke() | Select-Object Device, Model, IsSSD,
Size

# vSAN eligibility (vdq has no esxcli equivalent -- use Get-VsanDisk for the
queried-state view)
Get-VsanDisk -VMHost $VMHost | Select-Object Name, IsSsd, IsCacheDisk,
CapacityGB

# vSAN storage layout
$esx.vsan.storage.list.Invoke() | Select-Object Device, IsSSD, InCmmds,
IsCacheDisk

# SATP rules (read)
$esx.storage.nmp.satp.rule.list.Invoke() | Where-Object { $_.Options -match
'enable_ssd' }
```

Note: *`partedUtil`, `vdq`, and the SATP `rule add` / `reclaim` mutations are best run from the ESXi shell — PowerCLI proxies for them are inconsistent across versions. The view-style calls above are reliable.*

Maintenance Mode

```
# bash on ESXi host via SSH (appliance-only)
# Enter maintenance mode
esxcli system maintenanceMode set -e true -m noAction

# Exit maintenance mode
esxcli system maintenanceMode set -e false

# Check maintenance mode status
esxcli system maintenanceMode get
```

```
# PowerShell on Windows workstation via PowerCLI
Connect-VIServer vcenter.lab.local -User 'administrator@vsphere.local' -
Password '<password>'

# Enter MM (vSAN: choose -VsanDataMigrationMode Full / EnsureAccessibility /
NoAction)
Set-VMHost -VMHost (Get-VMHost esxi01.lab.local) -State Maintenance `
```

```
    -VsanDataMigrationMode Full -Confirm:$false

# Exit MM
Set-VMHost -VMHost (Get-VMHost esxi01.lab.local) -State Connected -
Confirm:$false

# Check current MM state
Get-VMHost esxi01.lab.local | Select-Object Name, ConnectionState
```

> ***Note:*** *For vSAN clusters, the recommended migration mode for a host* ***rebuild*** *is* `Full` *(full data migration). See VCF Operations memory* `feedback_vsan_mm_full_data_migration.md` *.*

8.3 Windows Depot Server Commands

PowerShell Commands

```
# Navigate to depot directory
cd C:\VCF-DEPOT

# Generate certificate
python generate_cert.py

# Start HTTPS server
python https_server.py

# Install Python cryptography library
pip install cryptography
```

9. Troubleshooting Flowcharts

9.1 Offline Depot Connection Failure

```
START: "Secure protocol communication error"
  |
```

```
  ├─→ Test connectivity: ping <depot-ip>
  │     └─→ FAIL: Check network/firewall
  │
  ├─→ Test SSL: openssl s_client -connect <ip>:8443
  │     └─→ FAIL: Check depot server is running
  │
  ├─→ Check certificate: View cert details
  │     └─→ Wrong hostname: Regenerate certificate
  │
  ├─→ Import certificate to Java truststore
  │     └─→ keytool -import ...
  │
  ├─→ Verify fingerprints match
  │     └─→ MISMATCH: Re-import correct certificate
  │
  └─→ Restart LCM service
        └─→ Wait 2 minutes, retry connection
```

9.2 ESXi Certificate Mismatch

```
START: "Certificate doesn't match subject alternative names"
  │
  ├─→ Check current cert SAN
  │     └─→ openssl x509 -in /etc/vmware/ssl/rui.crt ...
  │
  ├─→ Set correct hostname
  │     └─→ esxcli system hostname set --fqdn=<FQDN>
  │
  ├─→ Backup old certificates
  │     └─→ mv /etc/vmware/ssl/rui.* /etc/vmware/ssl/rui.*.bak
  │
  ├─→ Generate new certificates
  │     └─→ /sbin/generate-certificates
  │
  ├─→ Restart services
  │     └─→ services.sh restart
  │
  └─→ Update thumbprints in VCF
        └─→ Re-validate hosts in UI
```

9.3 vSAN SSD Detection Failure

```
START: "Found zero SSD devices for SSD cache tier"
  │
  ├─→ Check SSD status: esxcli storage core device list
  │     └─→ "Is SSD: false" → Continue
  │
  ├─→ Shut down ESXi VM in Workstation
  │
  ├─→ Edit VMX file
  │     └─→ Add: sata0:X.virtualSSD = 1
  │
  ├─→ Power on ESXi VM
  │
  ├─→ Verify SSD status
  │     └─→ Still false: Check VMX syntax
  │
  ├─→ Check vSAN eligibility: vdq -q
  │     └─→ "Has partitions" → Clean up partitions
  │
  └─→ Clean up old vSAN config
        ├─→ esxcli vsan storage remove -d <device>
        └─→ partedUtil delete ...
```

9.4 vCenter Deployment Stuck

```
START: vCenter deployment stuck at percentage
  │
  ├─→ Wait 30 minutes (large downloads may be slow)
  │
  ├─→ SSH to vCenter VM (ssh root@<vcenter-ip>)
  │     └─→ Default password: vmware
  │
  ├─→ Check firstboot status
  │     └─→ cat /var/log/firstboot/firstbootStatus.json
  │
  ├─→ Check for activity
  │     ├─→ vmstat 1 5 (disk I/O)
  │     └─→ tail -f /var/log/vmware/firstboot/installer.log
  │
  ├─→ If stuck at 60% "Installing Containers"
  │     ├─→ Check postgres: ls /storage/db/vpostgres/
  │     ├─→ Missing postgresql.conf → Database failed to init
  │     └─→ UNRECOVERABLE: Must redeploy
  │
```

```
   ├─→ Check services: vmon-cli --list
   │     └─→ Services not started → Check individual logs
   │
   └─→ If unrecoverable:
         ├─→ Delete vCenter VM
         ├─→ Clean up vSAN on all hosts
         ├─→ Reset depot connection
         └─→ Retry deployment
```

9.5 vLCM Host Seeding Failure

```
START: "Extraction of image from host failed"
   │
   ├─→ Check SSH status on ESXi host
   │     └─→ vim-cmd hostsvc/runtimeinfo | grep ssh
   │
   ├─→ SSH Disabled?
   │     ├─→ vim-cmd hostsvc/enable_ssh
   │     └─→ vim-cmd hostsvc/start_ssh
   │
   ├─→ Verify SSH on ALL hosts
   │     └─→ Repeat for esxi01, esxi02, esxi03, esxi04
   │
   └─→ Retry vCenter deployment
```

9.6 NSX Certificate Trust/SAN Failure (VDT)

```
START: VDT reports NSX cert FAIL (Trust or SAN)
   │
   ├─→ Check which check failed
   │     ├─→ SAN FAIL: Certificate missing hostnames/IPs
   │     └─→ Trust FAIL: Certificate root not in SDDC Manager keystores
   │
   ├─→ If SAN FAIL:
   │     ├─→ SSH to NSX Manager as root
   │     ├─→ Create OpenSSL config with all SANs:
   │     │     DNS.1 = nsx-vip.lab.local
   │     │     DNS.2 = nsx-node1.lab.local
   │     │     DNS.3 = nsx-manager.lab.local  ← SDDC Manager's registered FQDN
   │     │     IP.1 = 192.168.1.70 (VIP)
   │     │     IP.2 = 192.168.1.71 (node)
```

```
   │      ├─→ Generate cert: openssl req -x509 ...
   │      ├─→ Build JSON: python (avoid shell PEM escaping)
   │      ├─→ Import via API: POST /api/v1/trust-management/certificates?
action=import
   │      ├─→ Apply to node: ?action=apply_certificate&service_type=API&node_id=
<uuid>
   │      └─→ Apply to VIP: ?action=apply_certificate&service_type=MGMT_CLUSTER
   │
   ├─→ If Trust FAIL (after cert replacement):
   │      ├─→ SSH to SDDC Manager as vcf, then su - to root
   │      ├─→ Pull cert: openssl s_client ... > /tmp/nsx-root.crt
   │      ├─→ Import to VCF store: keytool -importcert ...
trusted_certificates.store
   │      ├─→ Import to Java cacerts: keytool -importcert ... cacerts
   │      └─→ Restart services: sddcmanager_restart_services.sh
   │
   └─→ Re-run VDT after ~5 minutes
          └─→ Expected: NSX cert checks all PASS
```

9.7 VCF Operations Health Adapter "No Data Receiving" Status

```
START: Infrastructure Health Adapter shows "no data receiving"
   │
   ├─→ Check adapter log (VCF Ops 9.x path):
   │      tail -100 /storage/log/vcops/log/adapters/
   │        VMwareInfraHealthAdapter/VMwareInfraHealthAdapter_55.log
   │
   ├─→ If "Unable to fetch access token for the SDDC manager":
   │      ├─→ Test SDDC Manager auth from VCF Ops node:
   │      │      curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
   │      │        -H "Content-Type: application/json" \
   │      │        -d
'{"username":"administrator@vsphere.local","password":"..."}'
   │      ├─→ If token returned: credential issue in adapter
   │      │      ├─→ UI → Administration → Integrations → SDDC Manager
   │      │      ├─→ If System Managed Credential enabled → click ROTATE
   │      │      ├─→ If Credential dropdown empty → uncheck System Managed,
   │      │      │      click +, create credential, select it
   │      │      ├─→ Click VALIDATE CONNECTION → confirm "valid"
   │      │      ├─→ Click SAVE
   │      │      └─→ Reboot VCF Ops appliance (adapter may not pick up
   │      │            new credential without full restart)
   │      └─→ If connection refused: check DNS/network/cert trust
   │
```

```
├─→ If "PKIX path building failed" for NSX:
│      ├─→ If NSX is powered off → expected, ignore
│      └─→ If NSX is running → See flowchart 9.8 below
│
├─→ If "vROPs is not configured with NTP server":
│      └─→ Configure NTP on VCF Ops appliance (cosmetic warning,
│            does not block data collection)
│
└─→ After fix: wait 10 min for 2 collection cycles
       └─→ Verify: adapter status changes to "Collecting" (green)
```

Key paths on VCF Operations 9.x

Item	Path
Adapter logs	`/storage/log/vcops/log/adapters/<AdapterName>/`
Main vcops logs	`/storage/log/vcops/log/`
Collector GC log	`/storage/log/vcops/log/collector-gc-*.log`
VCF Adapter log	`/storage/log/vcops/log/adapters/VcfAdapter/VcfAdapter_254.log`
VMware Adapter log	`/storage/log/vcops/log/adapters/VMwareAdapter/VMwareAdapter_63.log`
vSAN Adapter log	`/storage/log/vcops/log/adapters/VsanStorageAdapter/VsanStorageAdapter_257.log`

> ***Note:*** *VCF Operations 9.x does NOT use `/var/log/vmware/vcops/adapters/` — that path from older Aria Operations versions no longer exists. All adapter logs are under `/storage/log/vcops/log/adapters/`.*

9.8 VCF Operations NSX Integration — PKIX / Connection Failure

```
START: NSX adapter shows Warning, logs show "PKIX path building failed"
  │
  ├─→ Verify NSX is actually reachable from VCF Ops node:
  │     curl -sk https://nsx-vip.lab.local/api/v1/node/status | head -5
  │     ├─→ "No route to host" → NSX not ready (check load avg below)
  │     └─→ Returns JSON → NSX is up, proceed to cert fix
  │
  ├─→ If VIP (.70) unreachable but node (.71) responds:
  │     ├─→ NSX cluster VIP not online yet
  │     ├─→ Check load: curl -sk -u admin:'<pass>'
  │     │     https://<node>:443/api/v1/node/status | grep load_average
  │     ├─→ Load > 20 on 6 cores = still booting (normal in nested,
  │     │     can take 30-60 min after power-on)
  │     └─→ Wait for load < 20, VIP will come online automatically
  │
  ├─→ Import NSX cert into VCF Ops Java truststore:
  │     ├─→ openssl s_client -connect nsx-vip.lab.local:443 \
  │     │     -showcerts </dev/null 2>/dev/null \
  │     │     | openssl x509 -outform PEM > /tmp/nsx-cert.pem
  │     ├─→ Find truststore: java -XshowSettings:properties 2>&1
  │     │     | grep java.home
  │     │     → /usr/java/jre-vmware-17
  │     ├─→ keytool -importcert -alias nsx-vip \
  │     │     -file /tmp/nsx-cert.pem \
  │     │     -keystore /usr/java/jre-vmware-17/lib/security/cacerts \
  │     │     -storepass changeit -noprompt
  │     └─→ Reboot VCF Ops appliance
  │
  ├─→ Fix NSX credential (two adapters to check):
  │     │
  │     ├─→ VCF section → nsx-vip.lab.local adapter:
  │     │     ├─→ System Managed Credential ROTATE rarely works for NSX
  │     │     ├─→ Uncheck System Managed Credential
  │     │     ├─→ Click + → create credential (admin / password)
  │     │     ├─→ Select credential, VALIDATE CONNECTION, SAVE
  │     │     └─→ If VIP still unreachable, wait for NSX load to settle
  │     │
  │     └─→ NSX section → Aria Admin adapter:
  │           ├─→ Points to nsx-manager.lab.local (node FQDN)
  │           ├─→ May connect even when VIP is down
  │           ├─→ Set credential (admin / password)
  │           ├─→ VALIDATE CONNECTION → SAVE
  │           └─→ This can start collecting before VIP comes online
  │
```

```
└→ Verify: All adapters show "Collecting" (green) in
      Administration → Integrations
```

> ***Key insight:*** *VCF Operations has TWO separate NSX adapters — one under the VCF Cloud Foundation account (uses VIP) and one under the standalone NSX section called "Aria Admin" (uses node FQDN). Both need valid credentials. The Aria Admin adapter can connect via the node FQDN even when the VIP is still offline after a fresh NSX boot.*

> ***Java truststore path on VCF Ops 9.x:*** `/usr/java/jre-vmware-17/lib/security/cacerts` *(password:* `changeit` *). The legacy* `/usr/java/jre-vmware/` *path does not exist.*

---

10. SDDC Manager Credential Cascade Failure

Symptoms:

- Password Management → Update Password, Rotate, or Remediate fails for NSX (or other component)
- Task history shows: `"Resources [nsx-vip.lab.local] are not available/ready"` or `"not in ACTIVE state"`
- Subsequent attempts fail with: `"Unable to acquire resource level lock(s)"`
- VCF Operations Fleet Management shows `"[2] account(s) has been disconnected"`
- SDDC Manager API `/v1/nsxt-clusters` shows empty or non-ACTIVE status
- Dozens of stuck IN_PROGRESS tasks accumulate (visible via `/v1/tasks?status=IN_PROGRESS`)

Root Cause Chain: A failed credential operation (often due to NSX being temporarily unreachable during a boot storm or maintenance) triggers a cascade:

1. NSX cluster resource gets stuck in `ACTIVATING` or `ERROR` state in `platform.nsxt` table
2. Stale exclusive locks remain in `platform.lock` table, blocking all new operations
3. Failed tasks remain as `IN_PROGRESS` in `platform.task_metadata` (resolved=false), piling up
4. Each retry from the UI creates more stuck tasks and locks
5. Even after NSX recovers, SDDC Manager won't attempt the operation because the status check fails prevalidation

Diagnosis:

```
# bash on workstation (or any host with curl + python3)
# 1. Get auth token from SDDC Manager
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<password>"}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['accessToken'])")

# 2. Check NSX cluster resource state (look for status field)
curl -sk "https://sddc-manager.lab.local/v1/nsxt-clusters" \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
# If status is "ACTIVATING" or "ERROR" instead of "ACTIVE" → this is the
problem

# 3. Check for stale resource locks
curl -sk "https://sddc-manager.lab.local/v1/resource-locks" \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
# Stale locks from failed operations will block all new operations

# 4. Check for stuck IN_PROGRESS tasks
curl -sk "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
  -H "Authorization: Bearer $TOKEN" | python3 -c \
  "import sys,json; d=json.load(sys.stdin); print(f'Stuck tasks:
{len(d.get(\"elements\",[]))}')"

# 5. Verify NSX is actually healthy (from SDDC Manager)
curl -sk -u admin:'<password>' --connect-timeout 10 \
  https://nsx-vip.lab.local/api/v1/cluster/status
# overall_status should be "STABLE"
```

```
# PowerShell on Windows workstation -- credential cascade diagnosis
$Sddc      = 'sddc-manager.lab.local'
$SddcUser  = 'administrator@vsphere.local'
$SddcPass  = '<password>'
$NsxVip    = 'nsx-vip.lab.local'
$NsxUser   = 'admin'
$NsxPass   = '<password>'

[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]'Tls12,Tls13'
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = {
$true }
```

```
# 1. Token
$tokenBody = @{ username = $SddcUser; password = $SddcPass } | ConvertTo-Json
$tokenResp = Invoke-RestMethod -Method Post -Uri "https://$Sddc/v1/tokens" `
    -ContentType 'application/json' -Body $tokenBody
$Token = $tokenResp.accessToken
$Hdrs  = @{ Authorization = "Bearer $Token" }

# 2. NSX cluster resource state
$nsxClusters = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/nsxt-
clusters" -Headers $Hdrs
$nsxClusters.elements | Select-Object id, status

# 3. Resource locks
$locks = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/resource-locks"
-Headers $Hdrs
"Resource locks: " + (($locks.elements | Measure-Object).Count)
$locks.elements | Format-Table -AutoSize

# 4. Stuck IN_PROGRESS tasks
$inProg = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/tasks?
status=IN_PROGRESS" -Headers $Hdrs
"Stuck tasks: " + (($inProg.elements | Measure-Object).Count)

# 5. NSX cluster status (direct from NSX manager VIP)
$pair = "$($NsxUser):$($NsxPass)"
$auth = 'Basic ' +
[Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes($pair))
$nsxStatus = Invoke-RestMethod -Method Get `
    -Uri "https://$NsxVip/api/v1/cluster/status" `
    -Headers @{ Authorization = $auth } -TimeoutSec 10
"NSX overall_status: $($nsxStatus.mgmt_cluster_status.status) / control:
$($nsxStatus.control_cluster_status.status)"
```

> ***Note on field name:*** *The exact JSON field for the NSX overall cluster state varies between NSX 3.x (`mgmt_cluster_status.status`) and 4.x (`detailed_cluster_status.overall_status`). If the example above returns `$null`, dump the raw response with `$nsxStatus | ConvertTo-Json -Depth 8` and pick the right path for your NSX version — this script does* ***not*** *verify field names against your specific NSX build.*

Fix — Full Database Repair:

> ***WARNING:*** *Direct database manipulation is unsupported and should only be done in lab environments. Always back up before modifying.*

Step 1: Access PostgreSQL on SDDC Manager

SSH to SDDC Manager as `vcf`, then `su -` to root. PostgreSQL uses TCP on 127.0.0.1 (not Unix sockets), and the password may not be easily discoverable. Disable the psql pager to prevent `--More--` prompts from corrupting interactive shell sessions:

```
# bash on SDDC Manager appliance via SSH (appliance-only)
# Back up pg_hba.conf
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak

# Temporarily allow passwordless local connections
sed -i 's/scram-sha-256/trust/g' /data/pgdata/pg_hba.conf

# Reload postgres (no restart needed)
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Disable psql pager (CRITICAL for scripted/remote sessions)
export PAGER=cat
export PGPAGER=cat
```

> ***Appliance-only:*** *All `psql` / `pg_ctl` / `/data/pgdata/` operations run inside the SDDC Manager appliance —* ***no PowerShell sibling*** *(per Option B skip rules: `psql`, file edits in `/data/`, on-host postgres administration).*

Step 2: Fix the stuck resource status

The `nsxt` table status can be `ACTIVATING`, `ERROR`, or other non-ACTIVE values:

```
# bash on SDDC Manager appliance via SSH (appliance-only)
# Check current NSX resource status
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT id,
status FROM nsxt;\""

# Fix ANY non-ACTIVE status
```

```
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt
SET status = 'ACTIVE' WHERE status != 'ACTIVE';\""
```

Appliance-only: `psql` *runs on SDDC Manager — no PowerShell sibling.*

Step 3: Clear stale resource locks

```
# bash on SDDC Manager appliance via SSH (appliance-only)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT
count(*) FROM lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
lock;\""
```

Appliance-only: `psql` *runs on SDDC Manager — no PowerShell sibling.*

Step 4: Mark stuck tasks as resolved

The `task_metadata` table in the platform DB tracks task resolution state. Unresolved tasks (resolved=false) from failed operations accumulate and can interfere with new operations:

```
# bash on SDDC Manager appliance via SSH (appliance-only)
# Check unresolved task count
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT
resolved, count(*) FROM task_metadata GROUP BY resolved;\""

# Mark all unresolved tasks as resolved
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE
task_metadata SET resolved = true WHERE resolved = false;\""

# Clear task_lock table if any entries exist
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
task_lock;\""
```

Appliance-only: `psql` *runs on SDDC Manager — no PowerShell sibling.*

Step 5: Restore pg_hba.conf (CRITICAL — do not skip)

```
# bash on SDDC Manager appliance via SSH (appliance-only)
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Verify it's back to scram-sha-256
grep -c 'scram-sha-256' /data/pgdata/pg_hba.conf
# Should return 4 or more
```

Appliance-only: *Edits under* `/data/pgdata/` *and* `pg_ctl reload` *run on SDDC Manager — no PowerShell sibling.*

Step 6: Restart operationsmanager service

```
# bash on SDDC Manager appliance via SSH (appliance-only)
systemctl restart operationsmanager
# Wait 2-3 minutes for it to fully start
systemctl is-active operationsmanager
```

Appliance-only: `systemctl` *runs on SDDC Manager — no PowerShell sibling.*

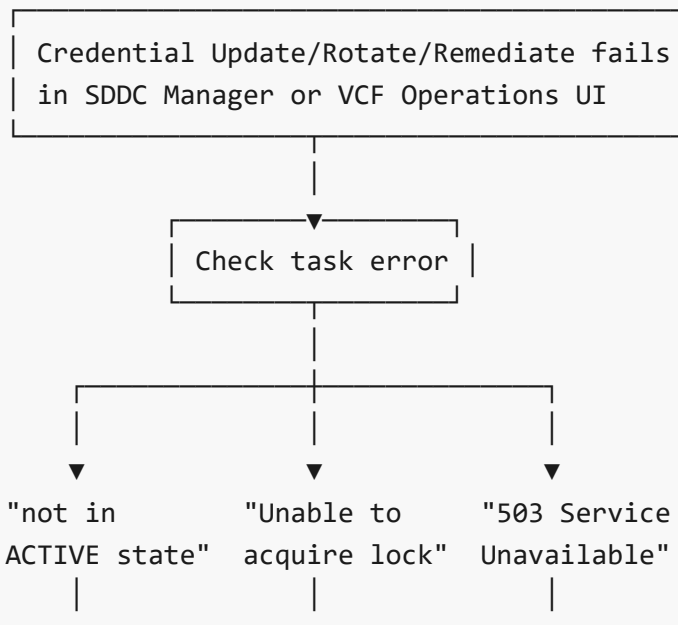
Verification:

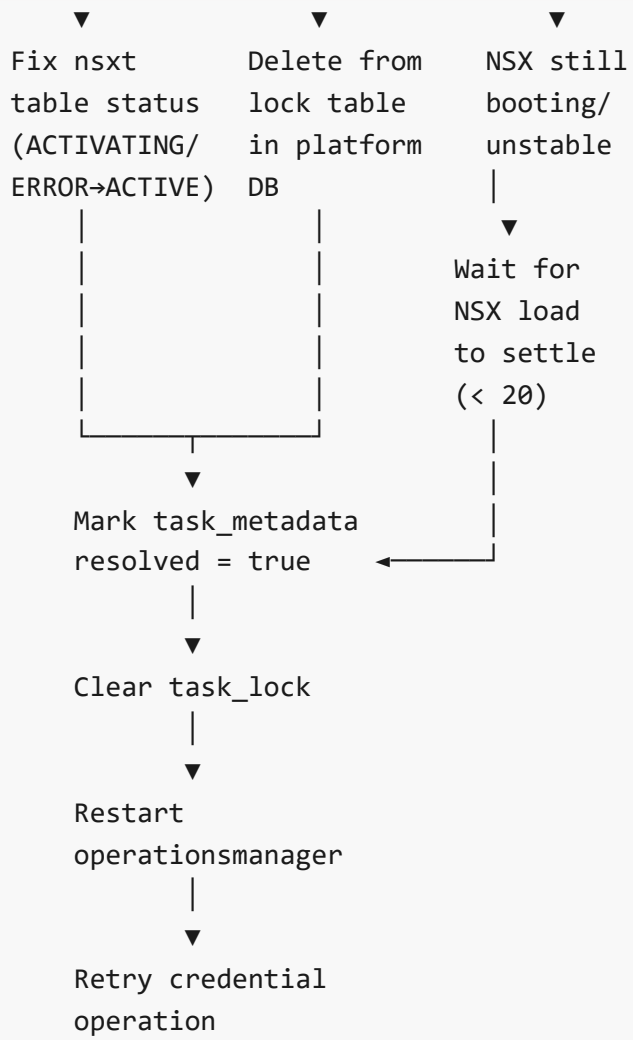
```
# bash on workstation (or any host with curl + python3)
# NSX cluster should now show ACTIVE
curl -sk "https://sddc-manager.lab.local/v1/nsxt-clusters" \
  -H "Authorization: Bearer $TOKEN" | python3 -c \
  "import sys,json; [print(f'{c[\"id\"]}: {c[\"status\"]}') for c in
json.load(sys.stdin).get('elements',[])]"

# Resource locks should be empty
curl -sk "https://sddc-manager.lab.local/v1/resource-locks" \
  -H "Authorization: Bearer $TOKEN"

# IN_PROGRESS tasks should be zero or minimal
curl -sk "https://sddc-manager.lab.local/v1/tasks?status=IN_PROGRESS" \
```

```
  -H "Authorization: Bearer $TOKEN" | python3 -c \
  "import sys,json; print(f'IN_PROGRESS:
{len(json.load(sys.stdin).get(\"elements\",[]))}')"

# Credential remediate should now succeed via VCF Operations Fleet Management
UI
```

```
# PowerShell on Windows workstation -- post-repair verification
# Reuses $Sddc / $Hdrs from the diagnosis block above (re-run those if the
token expired)

# NSX clusters: status should be ACTIVE
$nsxClusters = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/nsxt-
clusters" -Headers $Hdrs
$nsxClusters.elements | Select-Object id, status

# Resource locks: should be empty
$locks = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/resource-locks"
-Headers $Hdrs
"Resource locks: " + (($locks.elements | Measure-Object).Count)

# IN_PROGRESS tasks: should be zero or minimal
$inProg = Invoke-RestMethod -Method Get -Uri "https://$Sddc/v1/tasks?
status=IN_PROGRESS" -Headers $Hdrs
"IN_PROGRESS tasks: " + (($inProg.elements | Measure-Object).Count)
```

Credential Cascade Failure Flowchart:





Key insight: *Three tables in the `platform` database must be cleaned: (1) `nsxt` — resource status, (2) `lock` — operation locks, (3) `task_metadata` — task resolution tracking. The `operationsmanager` database has separate `task` and `execution` tables (columns: `task.state`, `execution.execution_status` — not `status`). The API won't let you cancel or delete stuck tasks — database repair is required.*

psql pager trap: *When running psql queries via Paramiko or remote shell, the default pager (`less` / `more`) captures output and waits for interactive input, corrupting the session. Always set `PAGER=cat` before running psql commands, or pass it inline: `PAGER=cat psql -h 127.0.0.1 -d platform -c "..."`. For Paramiko `invoke_shell()`, also set `height=1000` to prevent terminal-based paging.*

PostgreSQL on SDDC Manager: *Uses TCP on `127.0.0.1` (not Unix sockets — you'll get "No such file or directory" without `-h 127.0.0.1`). Data directory is `/data/pgdata`. Key databases:*

> *`platform` (nsxt, lock, task_metadata tables), `operationsmanager` (task, execution, processing_task tables). The `pg_hba.conf` trust workaround is a last resort — always restore the original immediately after.*

> ***vcf account lockout:** Failed SSH attempts (including from automated scripts) can lock the `vcf` account. SDDC Manager uses `faillock` (not `pam_tally2`). Unlock from console as root: `faillock --user vcf --reset`*

Discovery Process — How the Schema Was Mapped

None of the database repair procedure is documented by Broadcom. The schema was mapped through the following investigation:

Why the API wasn't enough:

- `PATCH /v1/tasks/{id}` with `{"status":"CANCELLED"}` returned `TA_TASK_CAN_NOT_BE_RETRIED` for every stuck task
- `DELETE /v1/tasks/{id}` returned HTTP 500
- The API has no mechanism to force-cancel stuck tasks or clear stale locks — database repair is the only path

How the database was explored:

1. Accessed PostgreSQL using the trust auth workaround (password not discoverable in config files)
2. Listed all databases with `\l`: `platform`, `operationsmanager`, `domainmanager`, `lcm`, `sddc_manager_ui`, `postgres`
3. Listed tables in `platform` DB with `\dt`: found `nsxt`, `lock`, `task_metadata`, `task_lock` (plus vcenter, host, etc.)
4. Queried column definitions via `SELECT column_name, data_type FROM information_schema.columns WHERE table_name = '<table>'`
5. Discovered that `task_metadata` uses a `resolved` boolean (not a `status` field like you'd expect)
6. Discovered that `operationsmanager.task` uses column `state` (not `status`) and `execution` uses `execution_status` (not `status`)
7. Early script versions failed because they referenced `task` in the platform DB (wrong — it's `task_metadata`) and used `status` column on operationsmanager tables (wrong — it's `state` and `execution_status`)

Why each repair step is needed:

Step	Table	Action	Why
2	`nsxt`	Set status to ACTIVE	The stuck ACTIVATING/ERROR status makes every new credential operation fail at prevalidation — SDDC Manager checks this before even attempting the operation
3	`lock`	Delete all rows	Stale exclusive locks from the failed operation block all new operations from acquiring their own locks — they'll fail with "Unable to acquire resource level lock(s)"
4	`task_metadata`	Set resolved=true	Unresolved tasks (resolved=false) accumulate with each UI retry. 47 were found during the initial diagnosis. These can interfere with new task scheduling
4	`task_lock`	Delete all rows	Links tasks to locks — clearing this ensures no orphaned task-lock relationships remain
5	`pg_hba.conf`	Restore backup	Leaving trust auth enabled means any local process can access PostgreSQL without a password — security risk
6	`operationsmanager`	Restart service	The service caches database state in memory. A restart forces it to re-read the cleaned tables and reset its internal state machine

Why each step matters in sequence:

- Steps 2-4 must all be done in one session — fixing just the status without clearing locks still fails, and clearing locks without fixing the status still fails. All three tables participate in the prevalidation check.
- Step 5 (restore pg_hba.conf) must happen before step 6 (restart service) because the service restart may trigger new database connections that should use proper authentication.
- The trust auth window should be as short as possible — do all queries, restore immediately, then restart.

Automated scripts built from this knowledge:

- `clear_locks.py` — fixes nsxt status + clears lock table (quick fix for simple cases)
- `fix_stuck_tasks.py` — marks task_metadata resolved + clears task_lock (for accumulated stuck tasks)
- `full_remediate_fix.py` — combines NSX health check + all DB fixes + service restart (all-in-one cascade repair)

All scripts automate the pg_hba.conf backup/trust/restore cycle and use `PAGER=cat` to prevent pager traps in remote sessions.

---

11. SDDC Manager Storage Migration (Local → vSAN)

Phase 7 — Feb 10–11, 2026

Why SDDC Manager Starts on Local Storage

This is a **chicken-and-egg bootstrap constraint** in every VCF deployment:

1. The VCF Installer OVA (which *is* the SDDC Manager appliance — same OVA, dual purpose) must be deployed **before** the bringup process runs
2. vSAN does not exist yet at this point — vSAN is created *during* the bringup process when the VCF Installer orchestrates the deployment of vCenter, vSAN, and VDS across the ESXi hosts
3. The only storage available before bringup is the **local datastore** on the ESXi host where you deploy the installer (`esxi01-local` in the lab)
4. After bringup completes, the VCF Installer **transforms into SDDC Manager** — still sitting on local storage where it was originally deployed

This means SDDC Manager is always initially deployed to local storage and must be manually migrated to shared storage (vSAN) afterward.

The Problem

- SDDC Manager was on `esxi01-local` with **914GB thick-provisioned** disks (only ~108GB actually used)
- Could not vMotion to other hosts — VM is on local storage, not shared
- esxi01 was overloaded — also running NSX Manager (32GB RAM) on the same host
- vCenter storage migration wizard **failed after 8+ hours** — it cannot thin-provision when migrating to vSAN

Disk analysis:

Disk	Allocated	Actual Used
sddc-manager.vmdk	32GB	2.6GB
sddc-manager_1.vmdk	16GB	2.6GB
sddc-manager_2.vmdk	240GB	3.0GB
sddc-manager_3.vmdk	512GB	99.5GB

sddc-manager_4.vmdk	26GB	30MB
sddc-manager_5.vmdk	88GB	64MB
Total	**914GB**	**~108GB**

Solution: vmkfstools Thin Clone

The vCenter migration wizard cannot thin-provision to vSAN. The workaround is to clone each disk individually as thin using `vmkfstools` directly on the ESXi host:

```
# bash on ESXi host (esxi01) via SSH (appliance-only)
# Clone each disk from local to vSAN as thin-provisioned
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager.vmdk -d thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_1.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_1.vmdk -d
thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_2.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_2.vmdk -d
thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_3.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_3.vmdk -d
thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_4.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_4.vmdk -d
thin

vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_5.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_5.vmdk -d
thin
```

> ***Appliance-only:*** *`vmkfstools` runs on the ESXi host shell against `/vmfs/volumes/` — no PowerShell sibling. PowerCLI's `Copy-HardDisk` / `New-HardDisk` cannot reliably thin-provision a clone to vSAN, which is exactly why this workaround exists.*

After cloning:

1. Power off SDDC Manager
2. Remove from vCenter inventory
3. Browse vSAN datastore → navigate to the `sddc-manager` folder → right-click the `.vmx` → **Register VM**
4. Power on and verify all services start (`systemctl status vcf-services`)

Recovery from Mid-Clone Failure

The 512GB disk clone failed partway through (ESXi connection timeout on nested storage). Fix:

```
# bash on ESXi host (esxi01) via SSH (appliance-only)
# Delete the partial clone
vmkfstools -U /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-
manager_3.vmdk

# Retry the clone
vmkfstools -i /vmfs/volumes/esxi01-local/sddc-manager/sddc-manager_3.vmdk \
  /vmfs/volumes/vcenter-cl01-ds-vsan01/sddc-manager/sddc-manager_3.vmdk -d
thin
```

Appliance-only: *`vmkfstools` runs on the ESXi shell — no PowerShell sibling.*

Key lesson: *vCenter's migration wizard cannot thin-provision to vSAN. Always use `vmkfstools -i <src> <dst> -d thin` per disk. This is also the only way to reclaim wasted space from thick-provisioned VCF appliances — SDDC Manager went from 914GB allocated to ~108GB actual on vSAN.* ***Broadcom reference:*** *KB 389960 — Migrating thick (eager/lazy zeroed) provisioned VMs to vSAN documents the OSR policy and RelocateVM.py workarounds; this section provides the verified `vmkfstools` path.*

---

12. Audit-Migrated Procedures (Formerly in `VCF-Undocumented-Issues-Reference` v5.x)

These 5 procedures were originally tracked in `VCF-Undocumented-Issues-Reference` as undocumented gotchas. As of the May 2026 audit, Broadcom has shipped KB articles documenting these issues —

they've been moved here for operational convenience. Each entry includes the Broadcom KB cross-reference so you can pull authoritative procedures, plus the verified-in-lab summary.

12.1 Fleet Management Cert Generator Produces Wrong SANs

Symptom: Fleet Management deployment wizard precheck fails with "*Certificate validation for component — The hosts in the certificate doesn't match with the provided/product hosts.*" The wizard's "Generate self-signed certificate" option produces a cert whose SAN entries don't match the node FQDN/IP.

Broadcom references:

- KB 412322 — VCF Automation 9 environment deployment fails with cert SAN mismatch
- KB 421072 — Fleet Manager CSR IP marked as DNS bug

Verified workaround (lab): Generate the cert manually with OpenSSL with the correct SANs (DNS + IP). Replace the wizard-generated cert before submitting the deployment.

```
# bash on SDDC Manager (SSH as root) or any Linux host with openssl
# Replace fleet.lab.local / 192.168.1.78 with your Fleet FQDN/IP
cat > /tmp/fleet-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = fleet.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = fleet.lab.local
```

```
DNS.2 = fleet
IP.1 = 192.168.1.78
EOF

openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/fleet.key -out /tmp/fleet.crt \
  -config /tmp/fleet-cert.cnf

# Verify SANs
openssl x509 -in /tmp/fleet.crt -noout -text | grep -A5 "Subject Alternative
Name"
# Expected: DNS:fleet.lab.local, DNS:fleet, IP Address:192.168.1.78
```

Then paste the generated `/tmp/fleet.crt` and `/tmp/fleet.key` into the Fleet Management wizard's certificate fields (use "Provide existing certificate" instead of "Generate self-signed").

12.2 VCF Operations 9.x Adapter Log Paths Changed

Symptom: Following legacy guides for VCF Operations adapter troubleshooting leads to log paths that don't exist. The legacy path is missing from the 9.x appliance.

Broadcom reference: TechDocs — Location of Log Files

Verified path (VCF Operations 9.0.1): `/storage/log/vcops/log/adapters/`

```
# SSH to VCF Operations appliance (root)
ls /storage/log/vcops/log/adapters/
# Per-adapter subdirectories appear here. The legacy /usr/lib/vmware-
vcops/user/log/adapter*
# paths from older guides are NOT present in 9.x.
```

12.3 VDT Not Pre-Installed on SDDC Manager — Download Required

Symptom: Need to run VDT (VMware Deployment Toolkit) for NSX cert/SAN validation or other VCF diagnostics, but `vdt` command not found on SDDC Manager.

Broadcom reference: KB 344917 — Using the VCF Diagnostic Tool (VDT) for vSphere

Verified procedure:

```
# Step 1: Download VDT from Broadcom KB 344917 to your workstation
#         (login required; download as vdt-*.zip)

# Step 2: SCP the zip to SDDC Manager
scp vdt-*.zip vcf@sddc-manager.lab.local:/tmp/

# Step 3: SSH and install
ssh vcf@sddc-manager.lab.local
su -
cd /tmp
unzip vdt-*.zip
cd vdt-*/
./vdt --help

# Step 4: Run a check (example: NSX cert SAN validation)
./vdt run nsx_certificates
```

Related VCF variant: KB 314610 — VDT for SDDC Manager covers the SDDC Manager-specific tool variant.

12.4 Suite-API Auth Header — Use `OpsToken` (Current) Instead of `vRealizeOpsToken` (Legacy)

Symptom: Scripts written against VCF Operations Suite-API using `Authorization: vRealizeOpsToken <token>` work but are using the deprecated header format.

Broadcom reference: TechDocs — Acquire an Authentication Token

Current format (VCF 9.0.1):

```
# bash on workstation — current header format
curl -sk -H "Authorization: OpsToken $TOKEN" \
  https://vcf-ops.lab.local/suite-api/api/...

# Legacy (still works for backward compatibility but deprecated):
# curl -sk -H "Authorization: vRealizeOpsToken $TOKEN" ...
```

PowerShell:

```
[Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
$hdr = @{ Authorization = "OpsToken $TOKEN"; Accept = 'application/json' }
Invoke-RestMethod -Uri 'https://vcf-ops.lab.local/suite-api/api/...' -Headers
$hdr
```

Migrate scripts to `OpsToken` before Broadcom removes the legacy alias in a future release.

12.5 Photon OS 5 "Desired State" Network Model — Static IP Lost on Reboot

Symptom: Manually configuring static IP on VCF Operations / Fleet Manager / other Photon OS 5 appliances (editing `/etc/systemd/network/10-eth0.network`) appears to work, but the appliance reverts to DHCP or a different IP on the next reboot.

Broadcom reference: KB 425535 — Static routes added to /etc/systemd/network do not persist

Root cause: Photon OS 5 appliances read vApp properties from vCenter on every boot and overwrite `/etc/systemd/network/10-eth0.network` to match. The vApp properties are the single source of truth.

Verified procedure — set vApp properties in vCenter (NOT in the guest OS):

```
vCenter UI:
1. Find the appliance VM in inventory (powered off recommended)
2. Right-click VM > Edit Settings > VM Options tab
3. Expand "vApp Options"
4. Verify "Enable vApp options" is checked
5. Under "OVF Settings", set OVF Environment Transport: VMware Tools
6. Click "Edit" next to "Properties"
7. Set the network properties to your desired static IP:
   - Network Gateway: 192.168.1.1
   - Network IPv4 Address: 192.168.1.77
   - Network Netmask: 255.255.255.0
   - Network DNS: 192.168.1.1
   - Network Domain: lab.local
   - Network NTP: pool.ntp.org
8. Save settings
9. Power on the VM — Photon OS reads the vApp props and writes them to
/etc/systemd/network/
```

Verify after first boot:

```
ssh root@vcf-ops.lab.local
cat /etc/systemd/network/10-eth0.network
# Should match the vApp properties set in vCenter
```

Do NOT edit `/etc/systemd/network/10-eth0.network` directly — the file is rewritten from vApp props on every boot.

Document Information

Field	Value
Document Title	VCF 9.0.1 Nested Deployment Troubleshooting Handbook
Version	1.7
Last Updated	May 11, 2026
Environment	VMware Workstation 17.x Nested Lab
VCF Version	9.0.1
Companion Reference	`../POWERSHELL-TRANSLATION-REFERENCE.md` — bash ↔ PowerShell translation patterns used throughout this handbook

Revision History (last 2 entries)

Version	Date	Change
1.6	May 2026	Earlier revisions
1.7	May 11, 2026	Added §11.3 Broadcom KB 389960 cross-reference for thin-provision-to-vSAN. Added new **Section 12 — Audit-Migrated Procedures** with 5 entries moved from `VCF-Undocumented-Issues-Reference` v5.x → v6.0 (Fleet cert generator, VCF Ops adapter log paths, VDT install, Suite-API OpsToken header, Photon OS 5 Desired State). All include Broadcom KB cross-references.

This handbook is intended for lab and educational purposes. Always consult official VMware documentation for production deployments.